



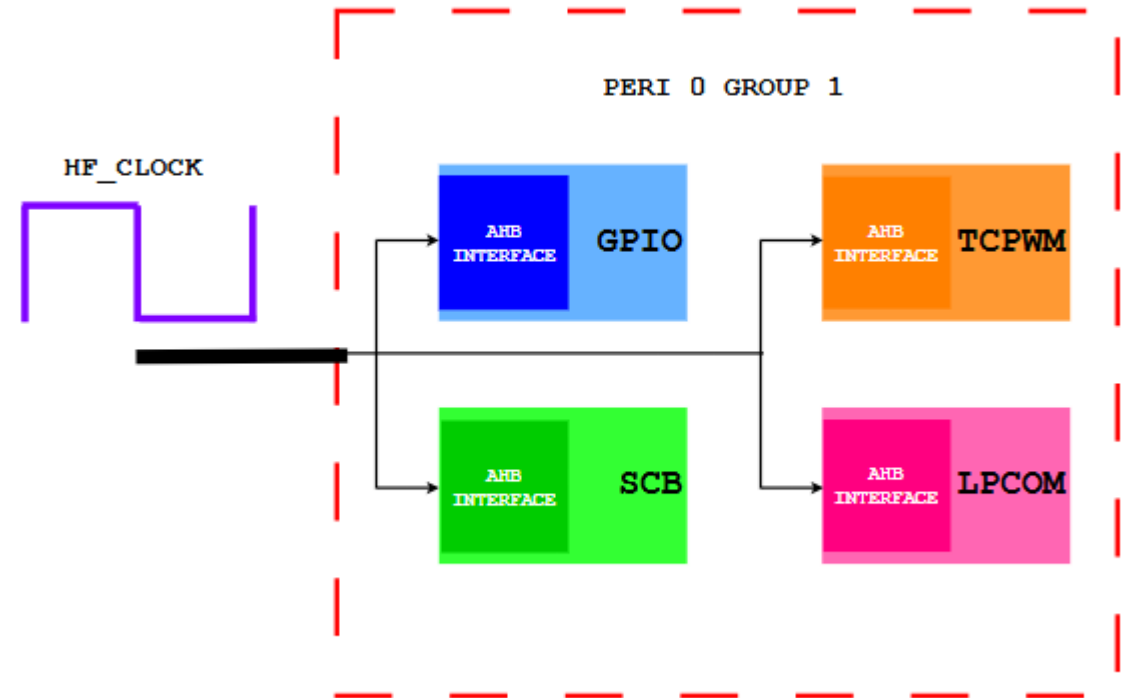
Zephyr: Power Domain Subsystem

Dylan Velasco



Power Domain - Zephyr

- Groupings of devices sharing a common source called power domain.
- Based on device runtime PM
- Applications using device A don't need to know about device B to manage their shared power domain.



Power Domain - Devicetree

- In the Device Tree, the power domain is represented as an abstract node.

hf_clock_domain_10:

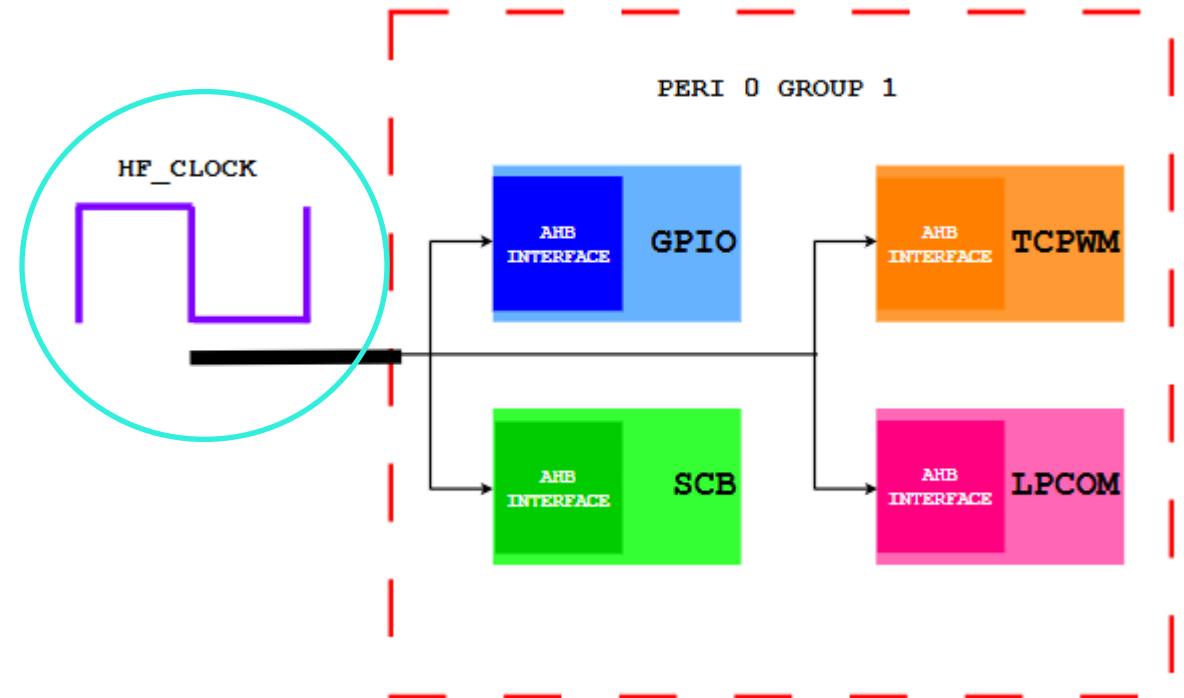
hf_clock_domain_10{

compatible = "power-domain";

#power-domain-cells = <0>;

zephyr,pm-device-runtime-auto;

};

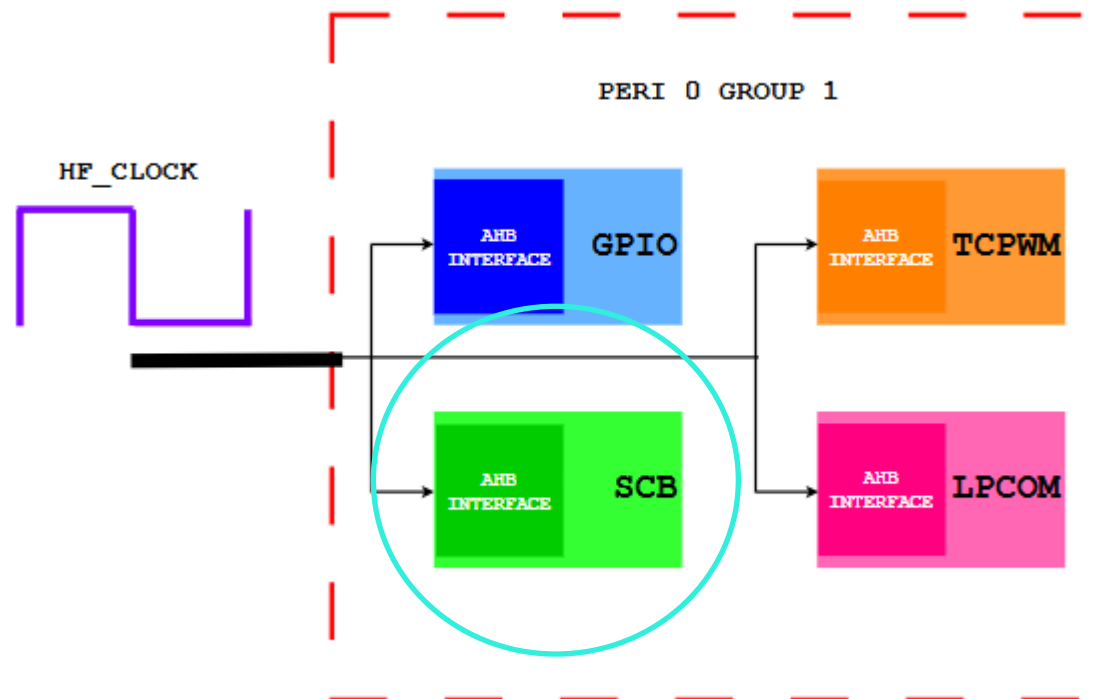


Power Domain - Devicetree

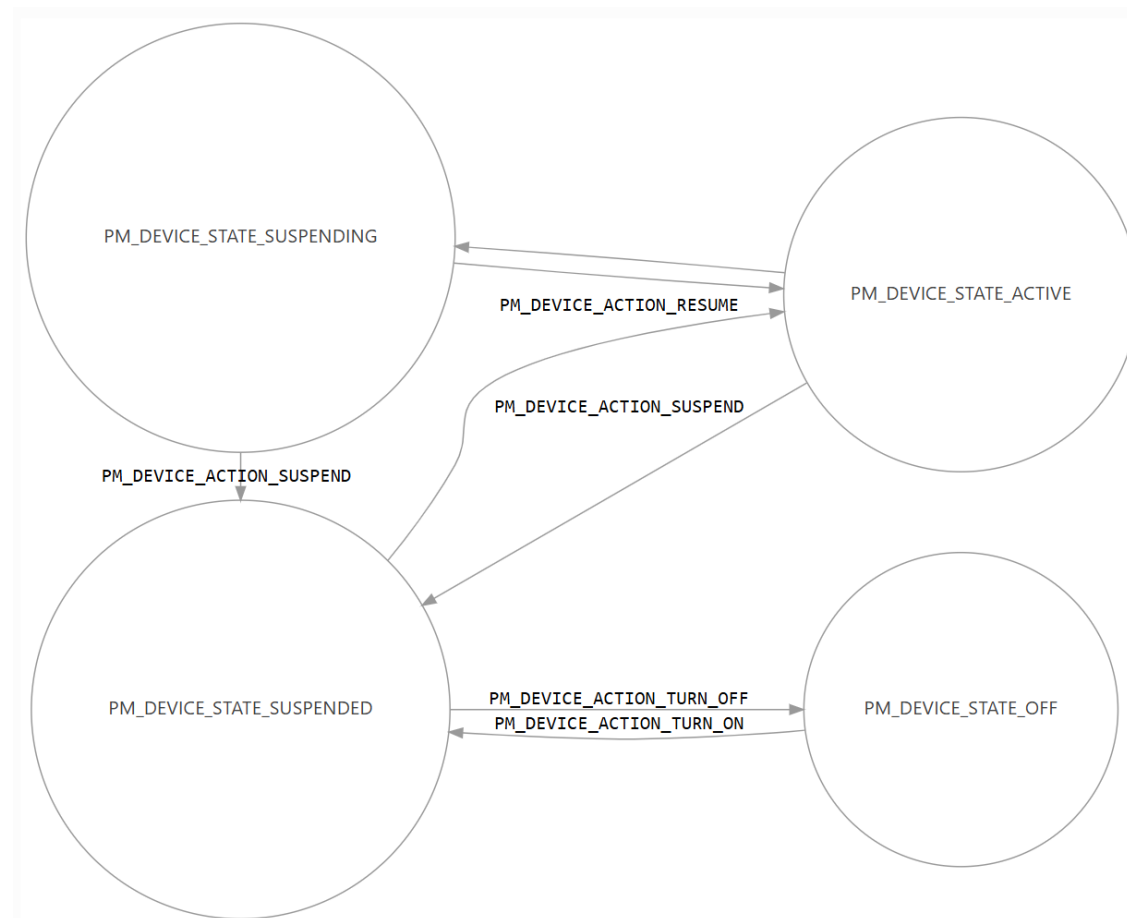
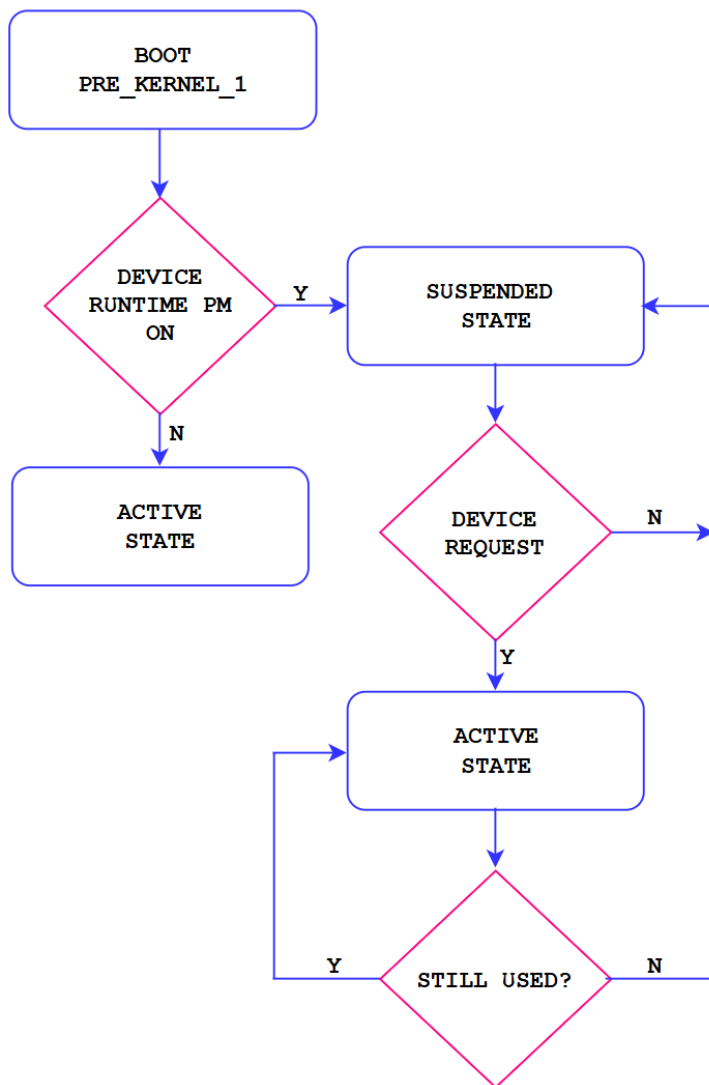
- Each device declares its power domain dependency in the Device Tree.

```
&scb2{
    power-domains = <&hf_clock_domain_10>;
    zephyr,pm-device-runtime-auto;
};
```

zephyr,pm-device-runtime-auto enables device runtime PM in that device.



Device PM – Working Flow

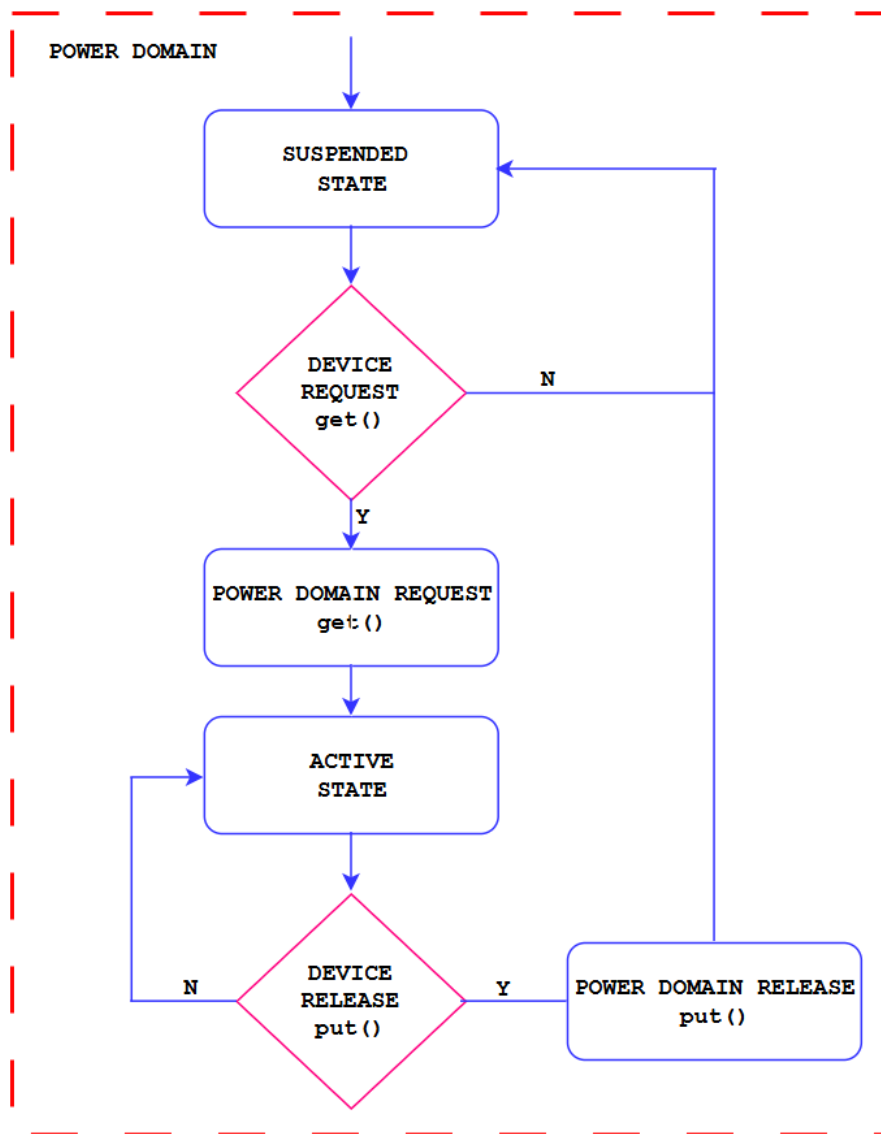


Device PM – Working Flow

- Device drivers indicate when they need the device to be operational and when they don't.
- The PM subsystem tracks usage via reference counting.
- The `get()` and `put()` functions within the driver indicate whether the device is needed, respectively.

Since the power domain is treated as another Runtime PM instance, the same flow and rules apply.

Power Domain – Working Flow



The power domain is automatically requested and released in response to `pm_device_runtime_get()` and `pm_device_runtime_put()` calls on the child devices

Power Domain - Driver

Since the power domain acts as an abstract entity that can be operated on, it requires a dedicated driver:

- Initialization function
- Callback function
- Device PM context
- Zephyr instantiation with pointer to the PM context:

```
static int clock_domain_pd_init(const struct device *dev)
{
    return pm_device_driver_init(dev, clock_domain_pd_action);
}

static int clock_domain_pd_action(*dev, pm_device_action action)
{
    switch (action) {
        case PM_DEVICE_ACTION_RESUME:
            /*resume the domain*/
            pm_device_children_action_run(PM_DEVICE_ACTION_TURN_ON);
        case PM_DEVICE_ACTION_SUSPEND:
            pm_device_children_action_run(PM_DEVICE_ACTION_TURN_OFF);
            /*suspend the domain*/
        case PM_DEVICE_ACTION_TURN_ON:
            /*turn on the domain*/
        case PM_DEVICE_ACTION_TURN_OFF:
            /*turn off the domain*/
    }
}
```

Power Domain - Driver

Since the power domain acts as an abstract entity that can be operated on, it requires a dedicated driver:

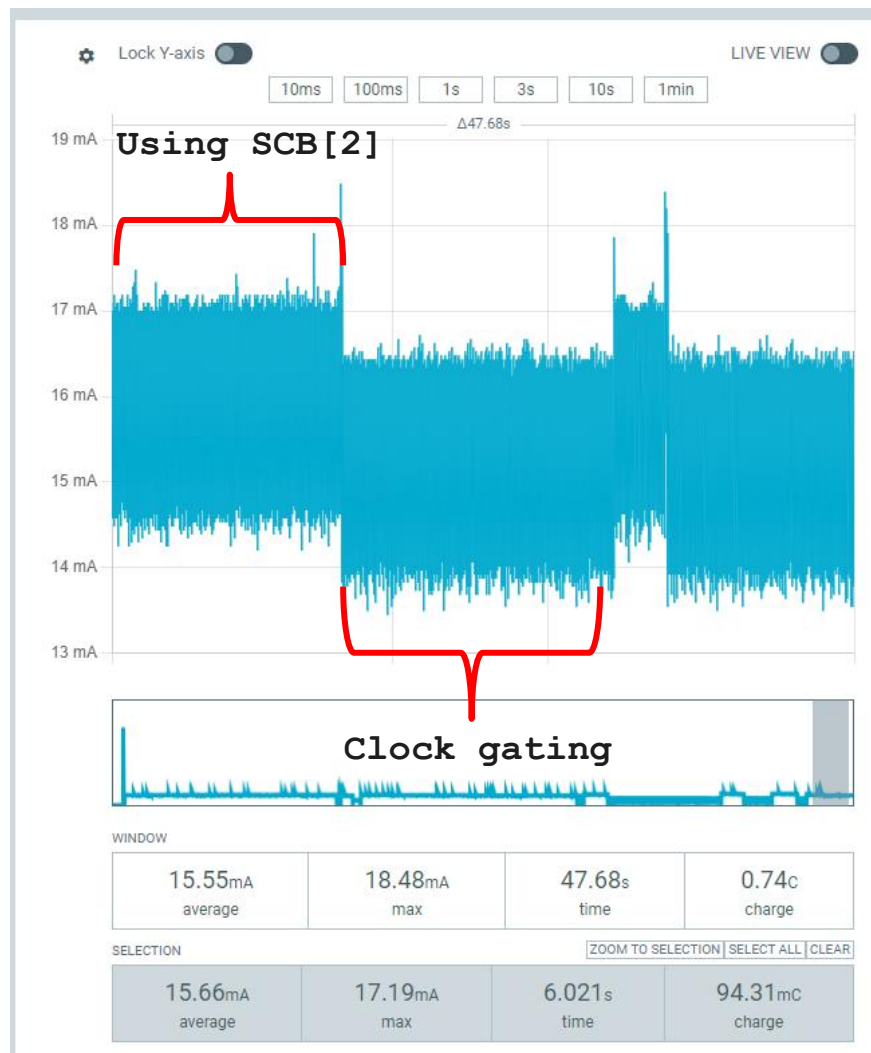
- Initialization function
- Callback function
- Device PM context
- Zephyr instantiation with pointer to the PM context:

```
#define INFINEON_CLOCK_PD_INIT(n)
```

```
PM_DEVICE_DT_INST_DEFINE(n, clock_domain_pd_action);  
DEVICE_DT_INST_DEFINE(n, &clock_domain_pd_init,  
                      PM_DEVICE_DT_INST_GET(n),  
                      NULL,  
                      NULL,  
                      PRE_KERNEL_1,  
                      11,  
                      NULL);
```

```
DT_INST_FOREACH_STATUS_OKAY(INFINEON_CLOCK_PD_INIT)
```

Power Domain - Zephyr



Conclusions

- The power domain mechanism automatically calls `get()` and `put()` on the domain when a child device is used.
- The power domain is an external entity linked to devices through the Device Tree.
- By implementing Runtime PM in the power domain's child devices, the power domain is used implicitly and the drivers require no additional changes.

Next steps

- Define and implement clock tree subdomains.
- Each HF clock can act as a subdomain of the domain defined by its clock path.

